实 验 报 告 · EXPERIMENT REPORT

基于 RAG 的 Git 知识库问答智能体

从 501 页 PDF 到可追溯引用的问答链路：切块、向量化、双路检索与工具调用

摘　要

本实验以《Pro Git》中文版（第 2 版，v2.1.80）官方 PDF 为唯一语料，构建了一个面向 Git 与版本控制问题的检索增强问答系统。实验按「语料预处理 → 结构化切块 → 向量化 → 双路检索 → 工具调用式问答」五段推进，全部代码为手写实现，仅依赖 pymupdf 与 numpy。最终从 501 页正文中抽出 805 个语义块（中位 718 字），生成 805 × 1024 的归一化向量矩阵，并实现语义向量检索与 BM25 关键词检索两套召回路径，由对话模型自主决定检索次数与检索词。

实验中记录了两类关键结果：其一，语义检索在同义改写的提问上明显优于 BM25，例如「rebase 和 merge 有什么区别」的首位命中相似度 0.767，并由第二命中（0.764）精确落到「变基 vs. 合并」小节；其二，对语料未覆盖的问题（如何撤销已推送的提交），系统能明确声明「知识库没有」，但同时暴露出模型仍会补充书外命令，说明单靠提示词约束不足，需要代码侧的审计兜底。

关键词：检索增强生成；语义切块；文本嵌入；BM25；工具调用；幻觉抑制

姓　　名	唐成纲	学　　号	2026011352
专　　业	计算机科学与技术	完成时间	2026 年 11 月
代码规模	5 个源文件、约 950 行 Python	语　　料	Pro Git 中文版 PDF，501 页

一、实验概述

1.1 实验目的

大模型在回答技术问题时有两类典型失败：一是答得笼统、无法核对；二是自信地编造命令与参数。检索增强生成（Retrieval-Augmented Generation，RAG）的思路是先从受控语料中检索原文，再让模型基于原文作答，从而把「凭记忆回答」变成「凭材料回答」。本实验的目的不是调用现成框架跑通一个 Demo，而是**从零实现整条链路，并在每一段上记录为什么这样选、实测结果如何**，据此判断 RAG 的实际边界在哪里。

具体目标有三项：（1）把结构规整的技术书转成可检索的语义块；（2）在同一语料上实现语义检索与关键词检索两条路径，用同一组查询做对照；（3）让问答环节具备可核验的引用与可检测的编造。

1.2 实验环境与依赖

项目	说明
运行环境	macOS，Python 3.13，项目内独立虚拟环境 `.venv`
第三方依赖	`pymupdf` 2.1（PDF 解析）、`numpy` 2.5（向量运算）；其余全部使用标准库
嵌入模型	`text-embedding-v4`，1024 维，OpenAI 兼容接口（`/embeddings`）
对话模型	`qwen-plus`，通过 `/chat/completions` 调用，`temperature = 0.2`
网络访问	仅使用 `urllib` 直连 HTTP 接口，未安装 `openai` / `requests` 客户端

1.3 系统总体结构

系统分为离线与在线两段。离线段一次性完成 PDF 解析、切块、质量检查与向量生成；在线段每次问答只做查询编码、检索与模型调用。这种划分的依据是：**预处理成本高但只付一次，检索与生成成本低但要付无数次**，因此把能提前算的都提前算掉。



模块	文件	职责

结构探查与质检	`inspect_pdf.py`
	`inspect_pdf2.py`
	`check_chunks.py`
		统计书签层级、字号分布、页边文本、块内字体；抽样复检切块质量

结构化切块	`src/build_chunks.py`	按字号重建标题层级，按小节分组打包，产出 `chunks.jsonl`
向量化	`src/embed.py`	批量调用嵌入接口，L2 归一化后写入 `embeddings.npy`
检索层（两路）	`vector_search.py` `search_bm25.py`	语义检索走内存矩阵内积；关键词检索走零依赖 BM25
问答智能体	`src/agent.py`	工具调用主循环、历史裁剪、答案参数审计
提示词	`prompts/system_prompt.md`	角色、依据、引用格式、命令规范、边界

二、语料准备与结构探查

2.1 语料选型

语料选择服务于实验目的，本实验设了三条标准：**答案可核验**——Git 命令有确定行为与权威文档，模型回答的对错能逐条比对，幻觉才可被度量；**提问方式贴近真实**——书中以中文叙述夹英文命令，与用户实际问法一致，不会出现「全文中文、提问全是英文术语」的错配；**结构可被程序解析**——只有字体、书签、段落结构规整的电子书，才谈得上按语义切块。据此选定《Pro Git》中文版第 2 版官方 PDF。

字节层与文本层体检

在写任何解析代码之前，先确认这份 PDF 不是「扫描图片」或「字体未嵌入导致乱码」。检查结果：20 个字体全部以 FontFile2 嵌入，且均带 ToUnicode 映射，中文可正确抽取；代码块使用 RobotoMono 等宽字体，是真正的文本而非图片；文件含 Outlines 书签树，具备按章节切分的技术前提。这一步的意义在于**排除掉整条链路根本不成立的可能**，避免后续调试时把「PDF 没有文本层」误判为「代码写错了」。

2.2 结构探查：四个预判被实测推翻

通用 PDF 切块流程通常包含「正则合并硬换行、按书签建目录树、统计高频页眉页脚、按字体区分代码」四步。用两个只读脚本遍历全书后发现，这四条在本文件上**全部不成立**，见下表。

通用做法（预判）	本书实测	对策略的影响
用正则合并 PDF 的硬换行	段落本身就是独立的 text block，换行只发生在 block 内部	取消全局合并，只在 block 内拼接：中文直接相连，英文单词之间补空格
用书签树建立章 / 节 / 小节三级目录	书签只有一级 14 条、二级 101 条，没有三级；但正文中存在 267 个 13pt 小节标题	改用字号重建层级：22pt 为章、18pt 为节、13pt 为小节，比书签更细
统计高频页眉页脚行后剔除	页边区域共 608 种不同文本，每种只出现 1 – 3 次，全部是页码，频次统计失效	改判据：字号为 9pt 且内容为纯数字 → 页码，直接丢弃
按 span 的字体判断代码块	RobotoMono 覆盖 22 万字符，正文中的英文单词也用它，按 span 判断会把正文误判为代码	改判据：一行内所有 span 均为等宽字体、且此类行占比 ⩾ 60% → 代码块

表 2　四处预判与实测的差异。第二列数据全部由探查脚本统计得到，未作估计。

这一步的意义

切块是 RAG 中唯一「错了就再也救不回来」的一步：噪声进了向量库，后续无论怎么调检索参数都滤不掉。因此这一步应当先测量、再写代码，而不是先套模板、再靠肉眼抽查兜底。四处差异中的任意一处若沿用通用做法，都会产出脏块——最直接的是按字体判断代码，会把大量夹有英文单词的正文误当成代码块。

2.3 正文范围的确定

全书 501 页并非都要入库。封面、目录、版权页、索引页三类内容是「导航」而非「知识」：它们只是把书里已有的标题再列一遍，入向量库后会在检索时反复命中同一批标题词，挤掉真正有解释力的段落。因此正文范围由书签自动定位——从一级书签「引言」（第 17 页）开始，到一级书签「索引」的前一页结束，覆盖第 17 – 497 页、共 14 章。**把范围交给书签而不是写死页码**，换版本或换印次时不用改代码。

三、结构化切块与质量检查

3.1 切块策略

切块的产物是「检索的最小单位」，它同时决定召回能否命中、命中后模型能否读懂。块过大，一个向量代表多个主题，相似度被稀释；块过小，语义被切断，模型拿到半句话无法作答。策略围绕「语义完整」与「长度可控」两个冲突目标展开，分四步：

1. **按字号重建层级**：block 最大字号映射为章（22pt）、节（18pt）、小节（13pt）；9pt 纯数字判为页码丢弃；整行等宽字体判为代码。
2. **按小节分组**：遇到标题即开新组并保存路径，同一小节不会被拆到两组，保证主题不串。
3. **组内打包**：累计超过 800 字收块，新块保留尾部约 120 字重叠；不足 150 字的碎块并入前块，防止「本节小结」独占一个向量位。
4. **拼接路径锚点**：块首加一行「《Pro Git》章 > 节 > 小节」，否则一个孤立的命令块无法与「变基」这类主题建立语义关联。

参数	取值	依据
MAX_CHARS（单块上限）	800	中文技术书一节通常 1 – 3 千字，800 字约可承载一个小节的核心论述
OVERLAP_CHARS（块间重叠）	120	约为上限的 15%，覆盖「定义 + 一句结论」的常见跨块结构
MIN_CHUNK_CHARS（最短块）	150	低于此长度的块多为小结或过渡句，独立成块几乎不可能被命中

3.2 切块结果

805
语义块总数

718
单块字数中位数（p90 = 868）

550
含代码块，占 68%

50.5 万
块内总字数（p17–497）

最长块 2171 字，超 1200 字者 9 块，全部是「Git 与其他系统」中 svn、hg、log 的超长终端输出，属内容本身长度，强行切分只会破坏示例，故接受。含代码块占 68%，符合 Git 类知识以命令示例承载的特征，也解释了 BM25 在含命令名查询上不吃亏。

3.3 三个已修复的缺陷

编号	现象	原因与修法
D1	「GitHub」一节的块，Markdown 围栏提前闭合	原文本身含三反引号。修法：围栏长度取「文本中最长连续反引号数 + 1」，嵌套不破坏结构
D2	单块长度堆到 2556 字，远超上限	「代码块不劈开」意味着它不触发收块。修法：仍不劈开，但允许它触发打包边界
D3	块内出现「Figure 3.1 …」等无主图注	图片未进入文本层，图注成为悬空描述。修法：正则 `^Figure\s+\d+\.` 整块丢弃

质检脚本扫描全量块：页码残留 0 块、目录点线残留 0 块、图注残留 0 块。这组「全零」是切块环节唯一可自动化的验收指标，也是后续排查检索效果时排除「脏数据」的依据。

四、向量索引构建

4.1 嵌入与批量策略

嵌入环节把 805 个文本块逐条送入 `text-embedding-v4`，得到 1024 维向量，最终存成 `805 × 1024` 的 float32 矩阵，约 3.1 MB，全量常驻内存。实现上使用 `urllib` 直接构造 HTTP 请求，而非引入 OpenAI 客户端，原因是链路只有两个端点（`/embeddings` 与 `/chat/completions`），多一个依赖并不会减少代码量，反而增加版本不确定性。

批量大小是本次实验中唯一「撞过墙」的参数：沿用 OpenAI 惯例的 32 条 / 批会直接返回 `400`，实测该接口单批上限为 10 条。这说明**「OpenAI 兼容」只保证请求与响应结构一致，不保证服务端限制一致**；把批量做成配置项、失败后按错误码排查，比记住某个具体数字更可靠。

4.2 归一化与检索的等价性

向量写入磁盘前先做 L2 归一化。归一化之后，两个向量的内积等于其余弦相似度：

$$\cos(q, d) = (q \cdot d) / (\|q\| \, \|d\|) = q' \cdot d' \quad ，其中 \; q' = q/\|q\| ，\; d' = d/\|d\|$$

这带来两点直接的工程收益：检索端只需一次矩阵乘法 `matrix @ q`，无需逐行除模长；查询向量与库内向量使用同一套归一化规则，相似度可直接横向比较，便于用固定阈值（如 0.75）判断「是否对题」。代价是写入前多做一次除法，只付一次，可以接受。

4.3 三项接口层决策

决策	理由
4xx 不重试，5xx 退避重试	401 / 403 / 404 属于鉴权、权限或模型名错误，重试只会放大失败；仅对服务端错误做最多 3 次指数退避
单条截断至 4000 字	接口上限为 8192 token，中文 token 密度高，4000 字留出足够余量；实测最长块 2171 字，不会触发截断
行序与 `chunks.jsonl` 严格一致	向量矩阵不存块 ID，检索返回的是行号，因此加载时用 `assert len(chunks) == matrix.shape[0]` 阻断错位

为什么不用向量数据库

805 个块上，一次 `805 × 1024` 的矩阵向量乘法在本机实测约 0.01 ms，本身就是精确最近邻，不存在召回损失。faiss 的 IndexFlatIP 结果与之完全一致，而 HNSW 一类近似索引的收益要到百万级规模才显现。在 805 块的规模上引入近似索引，等于用「可能漏召回 + 多一份调参与运维负担」换取一个用不到的速度提升，属于**负优化**。据此判断：当前用 numpy 暴力内积，块数涨到十万级时再换 HNSW。

需要强调的是，这一层的时间主要花在查询编码而非检索计算上：本机实测单次向量检索 0.01 ms、单次 BM25 全量打分 0.54 ms，而一次远程嵌入请求约 300 ms。**优化顺序应据此排列**——先减少检索轮次、复用查询向量，而不是去优化内积这一层。

五、双路检索与对照实验

5.1 语义向量检索

查询先经同一嵌入模型编码为 1024 维向量并归一化，再与库内矩阵做一次内积，取分数最高的 top-k。由于矩阵已在写入时归一化，内积即余弦相似度，索引结构就是一个内存矩阵，没有倒排表、没有聚类、没有图结构。返回值除分数与行号外，还附带块所属的章节路径与页码，供后续引用。

5.2 BM25 关键词检索

BM25 这一路刻意做成**零依赖**：不引入 jieba 等分词器，中文按相邻二字切分（bigram），英文与命令按符号切整词，只使用标准库的 `Counter` 与 `math`。评分沿用标准形式：

$$\text{score}(Q, D) = \Sigma_{t \in Q} \text{IDF}(t) \cdot [\, f(t,D) \cdot (k_1+1) \,] \,/\, [\, f(t,D) + k_1(1 - b + b \cdot |D|/\text{avgdl}) \,]$$

其中 $k_1 = 1.5$、$b = 0.75$；IDF 采用 $\log((N - df + 0.5)/(df + 0.5) + 1)$，保证非负。二字切分的优点是无需词典、对中文技术术语不敏感；代价是「的、了」这类高频二字组会带来噪声，需要靠 IDF 压低权重。实测平均每块 240 个检索词，全量打分 0.54 ms。

5.3 对照实验

用三个查询分别跑两路检索，观察 top-1 命中位置。三个查询对应三种典型问法：改写式提问（术语不在文中出现）、术语明确的概念比较、命令名明确的功能询问。

查询	BM25 top-1（分值）	向量 top-1 / top-2	判读
如何撤销已经推送的提交	附录 C › git push（13.89）	撤消操作 p57（0.703） 撤消操作 p55（0.682）	BM25 被「push」带偏；向量找到撤消章节，但两节都只讲本地操作
rebase 和 merge 有什么区别	变基 p99（10.56）	变基 p99（0.767） 变基 vs. 合并 p108（0.764）	向量双命中，第 2 条正是对比小节；BM25 停在章一级
git stash 有什么用	贮藏工作 p235（9.14）	附录 C › git stash（0.832） 贮藏与清理 p233（0.812）	两路都命中正确章节；命令名明确时 BM25 并不吃亏

表 3　双路检索对照。Q1 两个方法都没能解决，原因是书里确实没有写 revert 流程——这是语料覆盖问题，不是检索算法问题。

5.4 结果分析

（1）语义检索在同义改写上优势明确。Q2 中「区别」二字在书中并不成词，BM25 只能靠「变基」「merge」等字面命中；向量则把「变基」与「变基 vs. 合并」小节拉到前二，正是问答所需的两段。

（2）BM25 在命令名上更稳。Q3 中 `git stash` 是完整命令，英文整词切分天然精确匹配，且不受语义漂移影响；向量把附录 C 的命令条目排首位（0.832）也正确，但优势不明显。

（3）Q1 暴露的是语料上限，不是检索缺陷。两个方法都命中「撤消操作」「重写历史」，说明检索本身正常；但书中未提供撤销已推送提交的做法，任何检索都无法捞出不存在的内容。评估时把「检索失败」与「语料缺失」分开统计，才能判断该优化检索还是该换语料。

六、问答智能体

6.1 从「一次性拼接」到「工具调用」

最简单的 RAG 实现是「检索一次 → 把结果拼进提示词 → 生成答案」。它的缺陷在于检索词写死在代码里：用户问「怎么撤销已推送的提交」，系统就用这句话去检索，一旦检索词与书的表述不一致，模型拿到的是不相关材料，却仍被要求作答。本实验改为把检索封装成工具，交给对话模型自主调用：模型决定要不要查、查几次、用什么词查、返回几条。

```
messages = [system_prompt]
loop (最多 4 轮):
    resp  = chat(messages, tools=[search_git_docs])
    if resp 无 tool_calls:
        return resp.content                          # 直接作答
    记录 assistant 消息 (含 tool_calls)
    for call in resp.tool_calls:
        query, top_k = 解析参数(call)
        结果 = 向量检索(query, top_k) → 「[章节路径 | 页码 | 相似度] 原文」
        messages += {role: "tool", tool_call_id: call.id, content: 结果}
if 达到轮次上限:
    messages += {role: "user", content: "仅基于已有材料作答，不足则直说没有"}
    return chat(messages).content
```

循环里有两处关键设计：其一，**保留 assistant 的原始消息（含 `tool_calls`）**，否则后续 `role=tool` 消息会变成孤儿，接口直接拒绝；其二，**达到轮次上限时追加一条指令**，强制模型基于已有材料作答，避免它在材料不足时继续检索、陷入不收敛的循环。

6.2 提示词：把「不许编」写成可执行规则

系统提示词按「依据 / 引用 / 命令 / 边界」四组约束组织，重点不在措辞，而在每条都能被检验：

约束	具体写法与目的
依据	只用检索材料中的信息；材料没覆盖就说「知识库里没有相关内容」并指出最接近的章节。**禁用「通常」「一般来说」填补空缺**——这两个词是编造最常见的入口
引用	关键结论后标注出处，格式固定为「Git 分支 > 变基，p99」；单次回答不超过 4 处，避免句句挂来源导致引用贬值
命令	给完整可执行写法，并说明副作用：是否可逆、是否改写已推送历史、是否需要强制推送。三项缺一不可
相似度分档	0.75 以上视为对题；0.5 – 0.75 必须读原文确认，可能是相邻主题；0.5 以下换词重试或直说没找到。**把连续分数压成三档，模型才有可执行的动作**

此外在工具说明中加入了「参数边界」：代码块里出现的命令与长短选项，必须能在检索原文中找到；书中没有的新参数一律不得补充，需要提示更安全的做法时，只写「书中未涉及，建议查官方文档」，不写具体参数名。

6.3 四道防线

#	机制	防的是什么
1	提示词参数边界	模型用预训练记忆补齐它「觉得」更安全的参数
2	代码侧答案审计：正则抽出答案中所有长选项，逐个与本轮检索原文比对，缺失即告警	模型绕过提示词——这一层不依赖模型配合，是真正的兜底
3	轮次与 top_k 硬上限（4 轮 / 8 块）	检索不收敛的循环，以及上下文被一次撑爆
4	历史裁剪对齐 user 消息边界	裁剪把 assistant(tool_calls) 与 tool 消息拆开，导致请求被接口拒绝

四道防线的分层逻辑是：**提示词是软的，能被模型绕过；代码是硬的，挡得住**。因此不能把「不编造」的期望全押在提示词上，而要在链路上留一个不依赖模型的检查点。这也解释了为什么第 2 条被放在代码里而不是提示词里——它的价值恰恰在于「模型不知道自己在被检查」。

七、端到端实验结果

7.1 成功案例：书中覆盖度高的提问

提问「rebase 和 merge 有什么区别？」，检索命中「变基 p99」（0.767）、「变基 vs. 合并 p108」（0.764）、「附录 C › git rebase p495」。模型在此基础上组织出三段式答案：

- **本质区别**：merge 生成新的合并提交、保留两个分支的完整历史；rebase 把当前分支相对共同祖先的提交以补丁形式重放到目标基底上，形成线性历史。
- **历史形态对比**：以表格形式给出提交图形态、历史语义、共同祖先处理三行差异，并标注「据『变基 vs. 合并』节归纳」。
- **约束与风险**：主动指出 rebase 不可用于已推送的提交，因为会改写提交 SHA-1，导致他人本地历史与远程不一致。

三处引用均落到「章节 > 小节，页码」，且第三点与原文 p108 的警告一致——这不是模型「知道」的，而是从检索到的原文里读出来的。这说明当语料覆盖度足够时，工具调用式 RAG 能给出既有依据又有组织的答案。

模型答案摘录（节选，原文未经润色）

「merge 创建一个新的合并提交，保留两个分支各自的完整历史……rebase 则把当前分支相对共同祖先的修改以补丁形式重放到目标基底上，形成线性历史（Git 分支 > 变基，p99）。若 experiment 已推送到远程，再对其执行 rebase 会改写提交 SHA-1，导致他人本地历史与远程不一致（变基 vs. 合并，p108）。」

摘录中两处引用指向的正是检索命中的第 1、2 条块——答案与检索结果之间的对应关系可以直接核对，这也是本项目把「章节路径 + 页码 + 相似度」一并塞回上下文的原因：不只是给模型看，也是给人复核。

七、端到端实验结果（续）

7.2 失败案例：语料没有覆盖的提问

提问「如何撤销已经推送的提交？」，模型两次检索后命中「撤消操作 p55 – 57」与「重写历史 p247」。答案的定性部分是正确且诚实的：明确指出知识库里没有该问题的具体操作步骤，只给到「已推送的历史不应轻易重写」这一原则，并建议查阅官方文档。

> 但审计器同时报出两个问题参数：`--force`、`--hard` 未在本轮检索到的原文中出现，可能是模型凭记忆补充，请自行核对。人工复核确认：这两个参数确实不在本轮材料中，属于「提示词约束住了大部分、但没能完全挡住」的情形。
> 更进一步的缺口：答案还给出了一段 `git revert <commit>` 的示例，而 `revert` 这一命令同样不在语料中——审计器只匹配长选项，无法捕捉命令名，因此这条越界没有被自动发现。

7.3 结果归纳

维度	结论
可追溯性	答对时可精确到章节与页码；答不出时能说明最接近的章节，没有出现「假装知道」的整段编造
幻觉抑制	提示词挡住了大部分，但参数与命令名仍会越界；**代码审计是目前唯一不依赖模型配合的关卡**
检索质量	语义检索在同义改写上明显优于 BM25；命令名明确时两者相当
性能	检索计算在毫秒级，端到端延迟由嵌入与对话请求主导（每轮数百毫秒）

7.4 两个案例的对照含义

把两个案例并排放，可以看出这套系统真正的能力边界不在「模型多聪明」，而在「**语料覆盖到哪里**」：同一个模型、同一段提示词、同一条检索链路，遇到书里写透的主题给出的是可核对的三段式答案，遇到书里没有的主题则退化为「诚实地说没有 + 少量越界补充」。这提示后续所有优化都应先问一句——问题出在检索、出在语料，还是出在生成？三者的修法完全不同。

另一个值得记录的现象是：**模型「承认不知道」的能力，是靠提示词里的具体动作换来的**，而不是靠「要诚实」这种要求。提示词里写清了「材料没覆盖就说知识库里没有，并指出最接近的章节」，模型才有一个可执行的替代动作；如果只写「不要编造」，它最可能的行为仍然是硬答。

八、讨论

8.1 已确认的局限

1. **没有自动评测集**。目前的效果判断依赖三个示例的人工判读，「好到什么程度」无法量化，也无法在改动参数后确认是变好还是变坏。这是本项目最实质的短板。
2. **语料停在 Git 2.x 早期**。revert 的完整流程、`--force-with-lease` 等后续版本才普及的用法书中未写，问到只能回答「没有」。这不是缺陷，而是选型时明确接受的边界，但必须在使用中说清。
3. **两路召回没有融合**。BM25 与向量各跑各的，谁也没用上对方的强项。融合的关键难点在于两套分数量纲不可比（BM25 分值无上界，余弦相似度在 0 – 1），不能简单加权。
4. **切块按字符数而非 token**。中文与代码的 token 密度差异较大，同样 800 字，代码块占用的模型上下文可能是中文段落的两倍，导致上下文预算不可控。

8.2 关于「要不要用现成框架」

主流框架能直接提供文档加载器、切块器、向量库封装与链式编排。以本项目的规模估算，用框架可省下大约两百行代码，但也会把三处关键细节藏在封装里：嵌入接口的批量上限、向量入库前是否归一化、多轮历史的裁剪边界。而这三处恰恰是本次实验中真正出问题的地方——批量 32 条报 400、归一化决定阈值能否通用、裁剪点错位导致请求被拒。

据此的取舍是：**以学习原理为目标时手写更合适**，因为踩过的坑会变成可迁移的判断；**以稳定交付为目标时应使用框架**，因为这些问题社区已经解决过，重写只是重复付出成本。两者不是水平高低之分，而是目的不同。

8.3 后续工作

1. **建 30 题评测集**，其中 10 题是书中确实没有答案的题，分别统计「答对率」与「正确承认不知道的比例」——后者才是衡量幻觉抑制的关键指标。
2. **用 RRF（Reciprocal Rank Fusion）融合两路召回**：按名次而非分数合并，天然规避量纲不可比的问题，再交给模型。
3. **把审计范围从长选项扩展到命令名**，用 `git [a-z-]+` 形式一并比对，补上 7.2 暴露的缺口。
4. **切块长度改为按 token 计量**，使上下文预算与实际模型开销对齐。

8.4 若要交付给他人使用，还差三件事

1. **可配置化**。模型名、批量大小、轮次上限目前分散在代码与 `.env` 中，应统一到一处配置；调参数不该动代码，否则每次实验都要改源码、留不干净的 diff。
2. **最小回归用例**。固定 5 条问答（含 1 条书中无答案的），每次改动后跑一遍并比对引用是否仍正确——没有这一步，任何优化都可能是「修一处、坏一处」。
3. **边界提示前置**。「只回答 Git 与版本控制」目前只写在系统提示词里，用户看不到；应在交互入口就说明范围，否则越界提问会消耗检索与生成，还换来一句礼貌拒绝。

九、结论

本实验在 501 页技术文档上完整实现了一条可运行、可追溯的 RAG 问答链路，共产出 805 个语义块与 805 × 1024 的向量矩阵，并实现了语义检索与 BM25 两套召回路径与工具调用式问答智能体。三点结论：

1. **预处理阶段的价值远高于参数调优**。四个「通用做法不成立」的实测结论、三个已修复的切块缺陷，直接决定了后续检索有没有干净材料可用；而块一旦脏了，检索层无论怎么调都无法挽回。
2. **检索方法的选择取决于问法，不取决于先进程度**。语义检索在同义改写上占优，BM25 在精确命令名上稳健，两者短板恰好错位，这为下一步做融合提供了明确依据；而在 805 块规模上引入近似索引属于负优化。
3. **抑制幻觉必须靠代码，不能只靠提示词**。失败案例中模型仍写出了书外参数与命令，说明软约束只能降低概率；真正起作用的是那一道不依赖模型配合的参数审计。

最后一点感受是：这个项目最确定的收获不是「RAG 能用」，而是看清了它的**上限由语料和评测决定，而不由模型决定**。换更强的模型，不会让书里没有的 revert 流程凭空出现；但补一份新语料、加一套评测集，系统的实际能力边界就会被真实地推进一步。

复现方式

```
python src/build_chunks.py    # 生成 data/chunks.jsonl (805 块)
python src/embed.py           # 生成 data/embeddings.npy (805×1024)
python src/vector_search.py "如何撤销已经推送的提交"
python src/search_bm25.py    "git stash 有什么用"
python src/agent.py           # 交互式问答 (/debug 查看检索命中)
```

参考资料

[1] Chacon S, Straub B. *Pro Git*（第 2 版，中文版 v2.1.80）[M/OL]. Apress, 2014. 本项目语料，501 页.

[2] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond [J]. *Foundations and Trends in Information Retrieval*, 2009, 3(4): 333-389.

[3] Cormack G V, Clarke C L A, Büttcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods [C]. *SIGIR*, 2009: 758-759.

[4] 阿里云百炼. text-embedding-v4 模型与 OpenAI 兼容接口调用说明 [EB/OL]. 官方接口文档.